

The Icosahedron Experiment: What Actually Works in Practice

Reuben Brasher

March 22, 2026

Lecture 4

Empirical, Not Theoretical

This lecture is grounded in sustained experimentation rather than theory.

- ▶ Eleven structured working sessions were conducted.
- ▶ Multiple domains were explored including probability and systems.
- ▶ Iterative refinement occurred across attempts.
- ▶ Approaches were continuously compared.
- ▶ Failures were analyzed rather than ignored.
- ▶ Focus remained on what works in practice.

Purpose of the Experiment

The goal of the experiment was evaluation rather than exploration.

- ▶ Identify repeatable success patterns.
- ▶ Identify consistent failure modes.
- ▶ Distinguish illusion of correctness from real correctness.
- ▶ Move from anecdotal success to systematic method.
- ▶ Replace intuition with evidence.
- ▶ Establish operational principles.

Initial Expectations

Initial assumptions reflected common intuitions about LLMs.

- ▶ Provide a clear description of the task.
- ▶ Receive working code as output.
- ▶ Expect minimal iteration.
- ▶ Assume internal reasoning by the model.
- ▶ Expect outputs to be reliable.
- ▶ Treat prompting as sufficient.

Observed Reality

Actual behavior diverged significantly from expectations.

- ▶ Outputs were inconsistent across runs.
- ▶ Logical errors appeared frequently.
- ▶ Systems were brittle under small changes.
- ▶ Outputs appeared correct but were not.
- ▶ Internal reasoning was opaque.
- ▶ Small ambiguities caused large failures.

Core Insight from Early Failures

Early failures revealed a central principle about system behavior.

- ▶ Lack of constraints leads to variability.
- ▶ Ambiguity propagates through outputs.
- ▶ Models fill gaps with plausible logic.
- ▶ Surface fluency hides deeper errors.
- ▶ Prompting alone cannot ensure reliability.
- ▶ Structure is required for stability.

Failure Case 1: Procedural Logic

A simple generator exposed weaknesses in unstructured interaction.

- ▶ Probability distributions were incorrect.
- ▶ Validation outputs were fabricated.
- ▶ Surface correctness masked deeper flaws.
- ▶ Reasoning was inconsistent across attempts.
- ▶ Outputs varied unpredictably.
- ▶ Results lacked reproducibility.

Diagnosis of Failure Case 1

The failure resulted from absence of structure rather than complexity.

- ▶ No explicit schema defined behavior.
- ▶ Domain rules were not grounded.
- ▶ Constraints were not specified.
- ▶ No intermediate validation occurred.
- ▶ Logic was inferred instead of defined.
- ▶ Ambiguity remained unresolved.

Failure Case 2: Hierarchical Complexity

A more complex system revealed deeper structural issues.

- ▶ Class hierarchies were inconsistent.
- ▶ JSON and code representations diverged.
- ▶ Tests failed due to mismatches.
- ▶ Representations became misaligned.
- ▶ Fragility increased with complexity.
- ▶ System coherence degraded over time.

Diagnosis of Failure Case 2

The primary issue was misalignment across representations.

- ▶ No unified schema linked layers.
- ▶ Representations evolved independently.
- ▶ Consistency was not maintained.
- ▶ No single source of truth existed.
- ▶ Errors amplified with complexity.
- ▶ Alignment was not enforced.

Transition: What Works Instead

Failures revealed a consistent pattern of success and failure.

- ▶ Unstructured generation produces failure.
- ▶ Structured transformation produces reliability.
- ▶ Variability decreases with constraint.
- ▶ Explicit steps improve control.
- ▶ Structure enables validation.
- ▶ The approach shifted to scaffolding.

Success Case: Layered Scaffolding

A structured workflow demonstrated stable performance.

- ▶ Natural language defined the domain.
- ▶ Concepts were expressed as tables.
- ▶ Tables were converted to JSON.
- ▶ JSON was mapped to typed models.
- ▶ Models produced implementation code.
- ▶ Code was validated with tests.

Key Insight from Success

Successful systems shared a common structural property.

- ▶ Each layer constrained the next.
- ▶ Ambiguity decreased step by step.
- ▶ Representations remained aligned.
- ▶ Errors were detected early.
- ▶ Outputs became predictable.
- ▶ Stability emerged from structure.

The Scaffolding Model

The experiment formalized a repeatable method.

- ▶ Priming defines domain and vocabulary.
- ▶ Grounding introduces structured representations.
- ▶ Decomposition breaks tasks into parts.
- ▶ Each stage constrains the next.
- ▶ Steps are sequenced explicitly.
- ▶ The process becomes repeatable.

Scaffolding Is Required

Scaffolding is necessary rather than optional.

- ▶ Without priming the domain is misunderstood.
- ▶ Without grounding ambiguity persists.
- ▶ Without decomposition complexity overwhelms.
- ▶ All components are required.
- ▶ Skipping steps reintroduces failure.
- ▶ Reliability depends on structure.

LLM as Pair Programmer

The model behaves similarly to a junior developer.

- ▶ It is capable but inconsistent.
- ▶ It requires supervision.
- ▶ It performs well with clear instructions.
- ▶ It struggles with ambiguity.
- ▶ It benefits from iterative feedback.
- ▶ It does not ensure correctness alone.

Role Mapping

Effective collaboration follows structured roles.

- ▶ The human defines direction.
- ▶ The model executes transformations.
- ▶ Roles may shift depending on context.
- ▶ Feedback is continuous.
- ▶ Validation remains human responsibility.
- ▶ Oversight is always required.

Practical Workflow

A consistent workflow emerged from repeated sessions.

- ▶ Define the schema explicitly.
- ▶ Generate structured representations.
- ▶ Produce implementation code.
- ▶ Generate validation tests.
- ▶ Review outputs carefully.
- ▶ Refine iteratively.

Critical Constraint

Large tasks must be decomposed into smaller steps.

- ▶ Complexity overwhelms the model.
- ▶ Errors compound without visibility.
- ▶ Validation becomes impossible.
- ▶ Decomposition enables control.
- ▶ Smaller steps improve accuracy.
- ▶ Structure enables correctness.

Connection to System Design

The findings align with established engineering principles.

- ▶ Systems require explicit structure.
- ▶ Boundaries reduce complexity.
- ▶ Representations must align.
- ▶ Determinism improves reliability.
- ▶ Separation of concerns is essential.
- ▶ Architecture governs behavior.

Implications for Developers

Benefits vary depending on experience level.

- ▶ Experienced developers gain productivity.
- ▶ They can evaluate outputs effectively.
- ▶ They understand decomposition.
- ▶ They recognize subtle errors.
- ▶ They enforce constraints.
- ▶ They control system behavior.

Risks for Novices

Inexperience increases the likelihood of failure.

- ▶ Over-trust in generated outputs occurs.
- ▶ Logical errors go undetected.
- ▶ Workflows remain unstructured.
- ▶ Plausible results are misinterpreted.
- ▶ Systems become fragile.
- ▶ Reliability decreases significantly.

Required Skills

Effective use requires new engineering competencies.

- ▶ Prompt structuring is necessary.
- ▶ Representation design is critical.
- ▶ Task decomposition is required.
- ▶ Output evaluation is essential.
- ▶ Iterative refinement is standard.
- ▶ Skills focus on engineering discipline.

Synthesis of the Experiment

The experiment leads to a clear conclusion.

- ▶ LLMs are not reliable by default.
- ▶ Reliability comes from structured workflows.
- ▶ Scaffolding stabilizes behavior.
- ▶ Engineering replaces prompting.
- ▶ Systems outperform ad hoc interaction.
- ▶ Structure determines success.

Transition to Phase 3

Even structured systems encounter limitations at scale.

- ▶ Failures still occur under complexity.
- ▶ Scaling introduces new risks.
- ▶ Reliability becomes systemic.
- ▶ Errors become harder to detect.
- ▶ Systems evolve over time.
- ▶ Next focus is failure at scale.

References I